

DCA0305

Lesson 11

morsinaldo.medeiros.075@ufrn.edu.br
ivanovitch.silva@ufrn.br

AI Agents II · MCP

One standard plug between AI applications and the world

Where we are

10

Agents I

The loop and tools, by hand

→

11

Agents II

MCP · Tools as a shared standard

→

12

Agents III

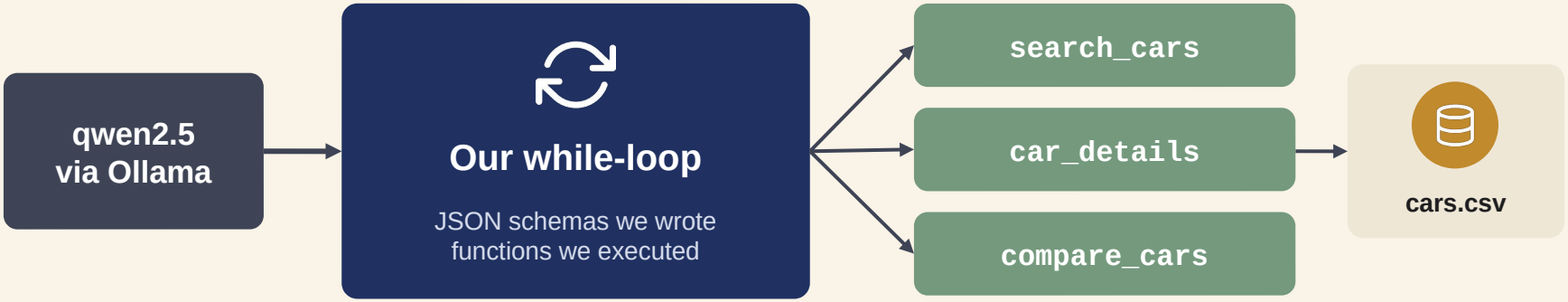
Frameworks, multi-agent

Why a protocol

The M×N problem · The USB-C idea

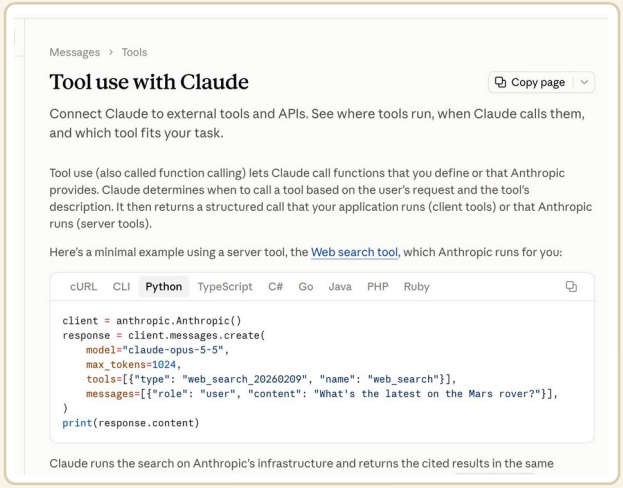
I

Last time: three tools, wired by hand

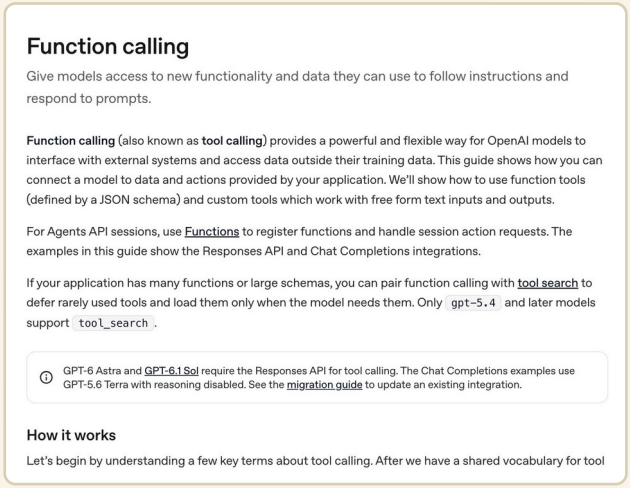


It works — for one agent, one model, one codebase. Now try sharing it.

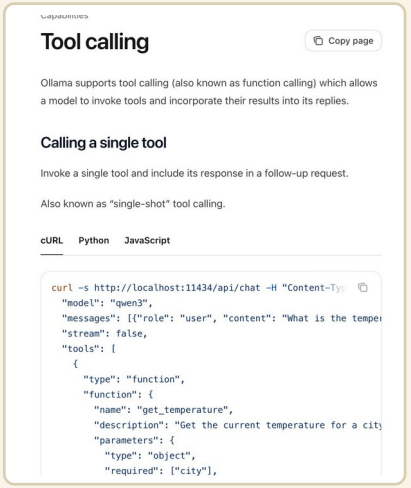
Now share those tools



Anthropic · tool_use blocks



OpenAI · function calling

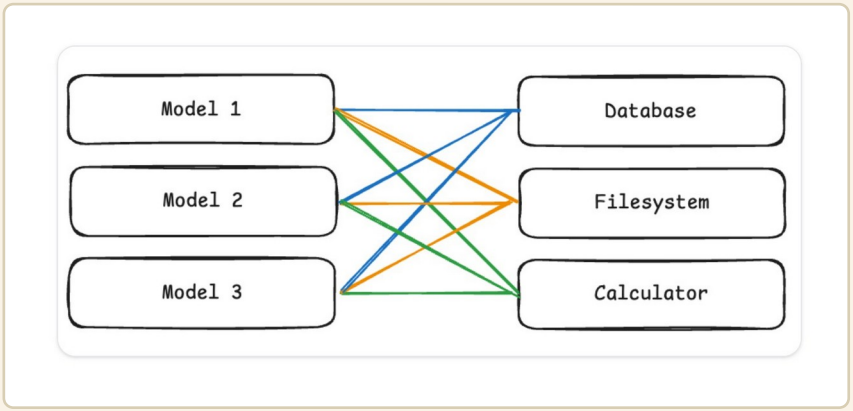


Ollama · "tools" field

Every new app or new model means new glue code.

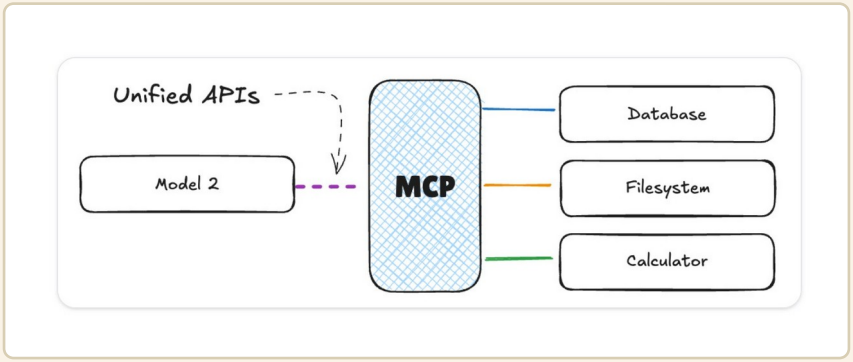
The M×N problem

Without a standard



M × N = **9 integrations**

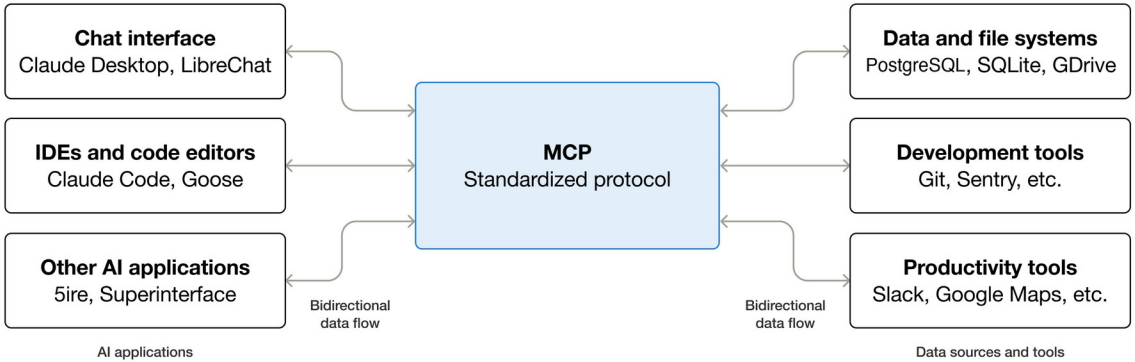
With MCP



M + N = **6 integrations**

figures: Hugging Face MCP Course — Key Concepts and Terminology

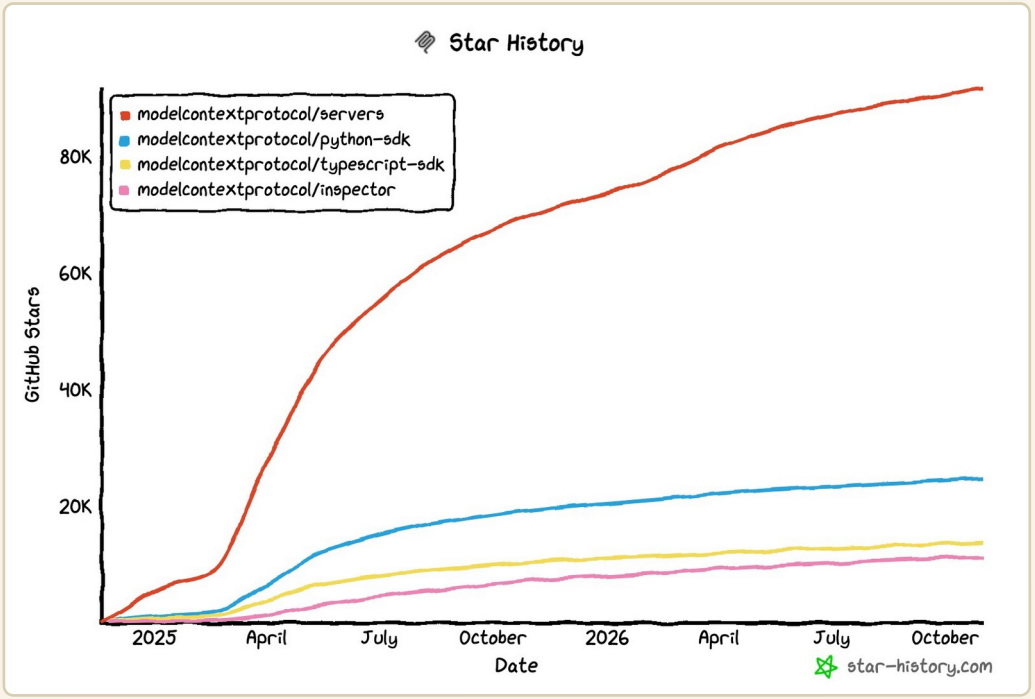
A USB-C port for AI applications



modelcontextprotocol.io — What is MCP?

Build a server once. Any compatible app can plug it in.

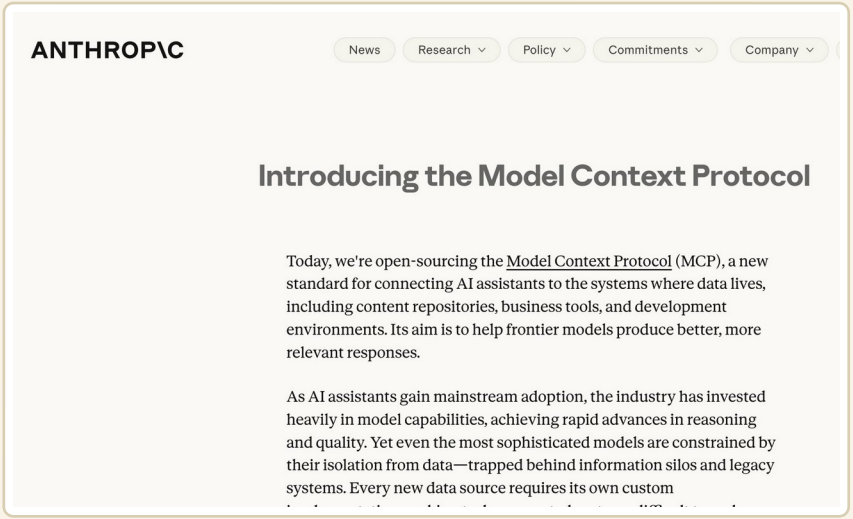
From internal tool to industry standard



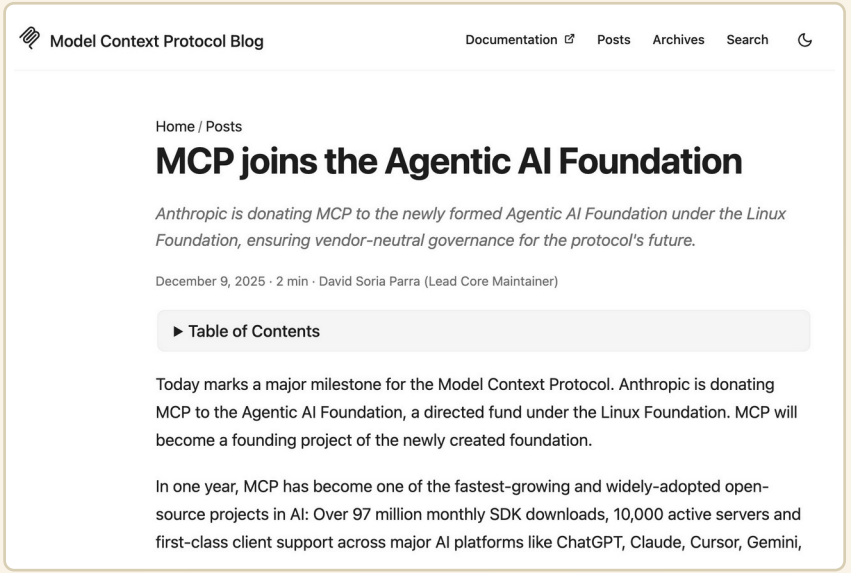
GitHub stars of the official repositories · star-history.com

- **Nov 2024**
  Anthropic open-sources MCP
- **Mar 2025**
  Streamable HTTP · OpenAI adopts it
- **Jun 2025**
  elicitation · structured output
- **Nov 2025**
  tasks · URL elicitation
- **Dec 2025**
  donated to the Linux Foundation
- **Jul 2026**
  stateless protocol

Two announcements, one year apart



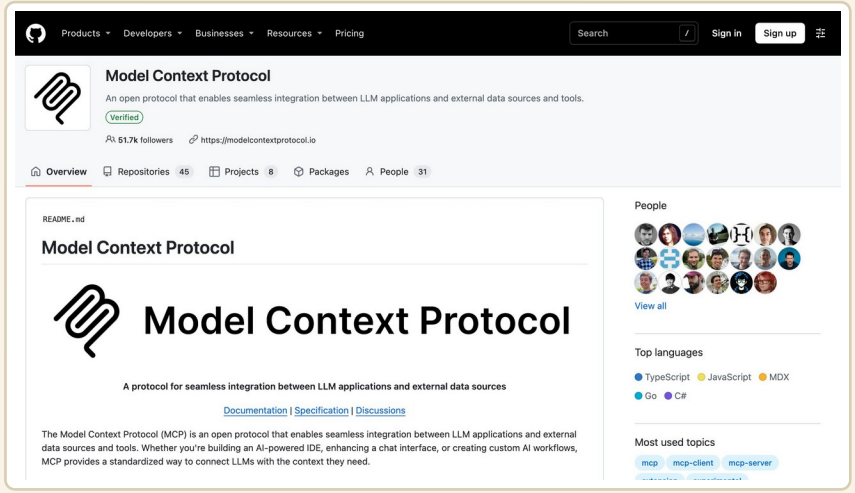
anthropic.com/news · Nov 25, 2024



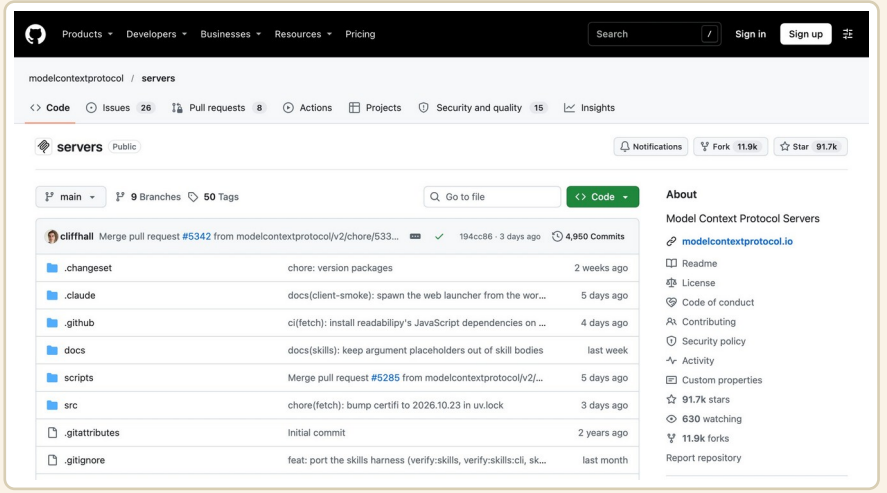
blog.modelcontextprotocol.io · Dec 9, 2025

From one company's project to a vendor-neutral standard.

All of it is open source



github.com/modelcontextprotocol



github.com/modelcontextprotocol/servers

Architecture

Host · Client · Server · Two layers

II

Three roles

Host

The AI app you use: Claude Desktop, Claude Code, VS Code, ChatGPT

Client

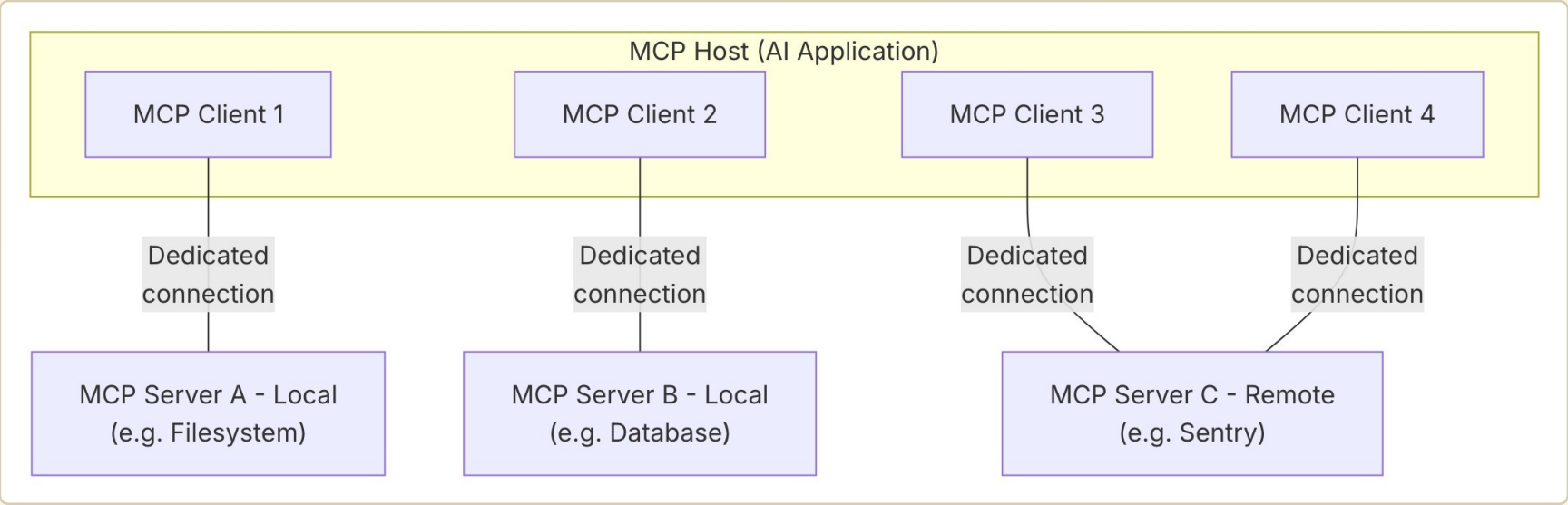
A connector inside the host — one per server, hidden from the user

Server

A program that exposes tools, data and prompts — local or remote

The host talks to the user and the LLM. Clients talk to servers.

One client per server



modelcontextprotocol.io — Architecture overview

Each connection is isolated: a server never sees another server — or its credentials.

Who does what

Host	Client	Server
Creates and manages clients	Speaks the protocol with ONE server	Exposes tools, resources, prompts
Asks consent, enforces permissions	Discovers what the server offers	Small and focused
Runs the LLM and the loop	Keeps the security boundary	Never sees the chat or other servers

Orchestration lives in the host. Servers stay easy to build.

Two layers

TRANSPORT LAYER

How bytes travel: stdio or Streamable HTTP · Framing · Authorization

DATA LAYER · JSON-RPC 2.0

Discovery	Primitives	Notifications
Versions · Capabilities	Tools · Resources · Prompts	Changes · Progress

Same JSON messages, whatever pipe carries them.

The protocol

JSON-RPC · Transports · Lifecycle

III

Remote procedure call

Call a function in another process as if it were local

```
# in the host
search_cars(
  80000, "SUV")
```

→

```
{"method": "tools/call",
 "params": {"name":
  "search_cars", ...}}
```

→

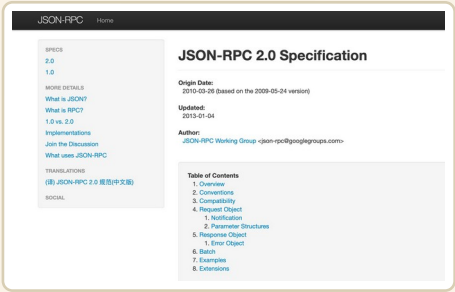
Server runs the real function

Travels as JSON

MCP picked JSON-RPC 2.0

Human-readable · Language-agnostic · Tiny

Older cousins: XML-RPC · SOAP · gRPC



jsonrpc.org/specification · 2010

JSON-RPC 2.0: three kinds of message

Request

Asks; expects an answer

```
{"jsonrpc": "2.0",
 "id": 7,
 "method": "tools/list",
 "params": {...}}
```

Response

Answers; same id

```
{"jsonrpc": "2.0",
 "id": 7,
 "result": {...}}
   or "error": {...}
```

Notification

One-way; no id, no answer

```
{"jsonrpc": "2.0",
 "method":
  "notifications/
   tools/list_changed"}
```

The id pairs each response with its request.

A real call, on the wire

Captured from our car server · Trimmed

→ request

```
{"jsonrpc": "2.0", "id": 3,
 "method": "tools/call",
 "params": {
   "name": "search_cars",
   "arguments": {
     "max_price": 80000,
     "body": "SUV"},
   "_meta": {...}}}
```

← response

```
{"jsonrpc": "2.0", "id": 3,
 "result": {
   "isError": false,
   "structuredContent": {
     "total_found": 22,
     "cars": [{"car_id":"c082",
       "model": "Duster", ...}]},
   "content": [{"type":"text",...}]}}
```

Same answer as lesson 10 (c082 on top) — **now over a standard wire.**

tools/list: a schema you did not write

```
{"name": "search_cars",
 "description": "Search the used-car
   stock by budget and body type...",
 "inputSchema": {
   "type": "object",
   "properties": {
     "max_price": {"type": "number"},
     "body": {"anyOf": [{"type": "string"},
              {"type": "null"}]}},
   "required": ["max_price"]}}
```

- **name** from the function name
- **description** from the docstring
- **inputSchema** from the type hints

Lesson 10’s hand-written JSON — now derived from Python.

Two kinds of error

Protocol error · The request is broken

```
{"id": 4, "error": {
   "code": -32602,
   "message": "Unknown tool:
     invalid_tool_name"}}
```

Unknown tool · Malformed request

Tool execution error · A result

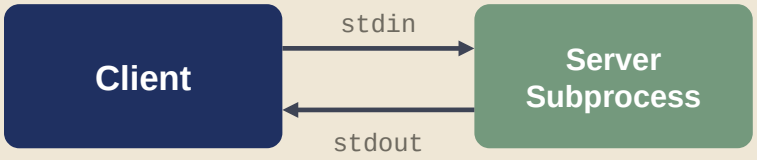
```
{"id": 4, "result": {
   "isError": true,
   "content": [{"text": "max_price:
    Input should be a valid number"}]}}
```

Real: we sent max_price = "barato"

Execution errors go back to the model so it can self-correct — lesson 10's "errors as observations", now in the spec.

Two transports, same messages

stdio · local



· Client launches the server process

· One JSON message per line

· Logs go to stderr, never stdout

Streamable HTTP · Remote



· One endpoint, many clients

· Answer as JSON or as a stream

· Auth over HTTP (OAuth)

The old HTTP+SSE transport (2024) is deprecated. **Use Streamable HTTP.**

Plugging a server into a host

```
// claude_desktop_config.json
{"mcpServers": {
   "estoque": {
     "command": "python",
     "args": ["server.py"]},
   "sentry": {
     "type": "http",
     "url": "https://mcp.sentry.dev/mcp"}
}}
```

```
# Claude Code
claude mcp add estoque \
  -- python server.py

claude mcp add \
  --transport http \
  sentry https://mcp.sentry.dev/mcp
```

stdio: a command to run

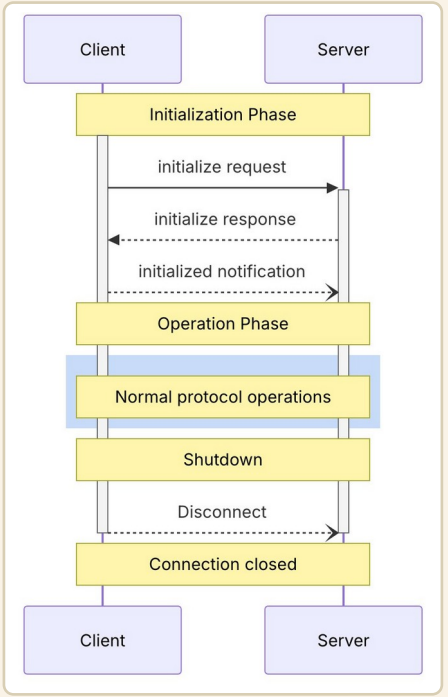
HTTP: a URL to reach

Until 2025: a handshake first

The lifecycle you will still find in most tutorials

- **`initialize`**
  Client proposes a version, both declare capabilities
- **`notifications/initialized`**
  The session starts
- **`operation`**
  List · Call · Notify, inside that session
- **`shutdown`**
  Close the pipe

Like a first date: agree on a version, show what you can do, then keep the line open.



Spec 2025-06-18 · Lifecycle

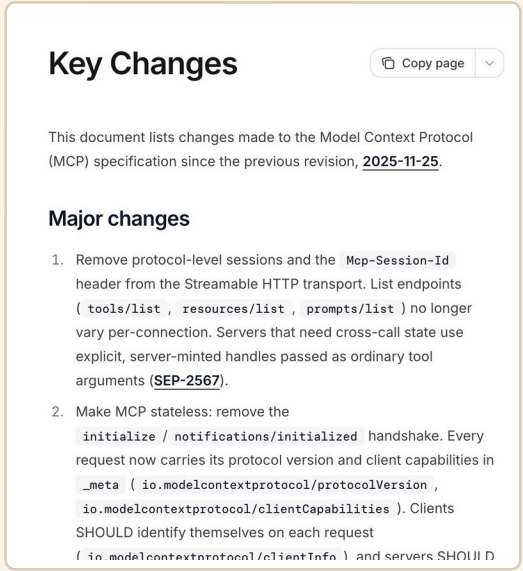
2026: every request stands alone

Protocol revision 2026-07-28

Real · Our server

```
→ {"method": "server/discover",
   "params": {"_meta": {
     ".../protocolVersion": "2026-07-28",
     ".../clientCapabilities": {}}}}

← {"supportedVersions": ["2026-07-28"],
   "capabilities": {"tools": {...},
     "resources": {...},
     "prompts": {...}}}
```



Key Changes

Copy page

This document lists changes made to the Model Context Protocol (MCP) specification since the previous revision, 2025-11-25.

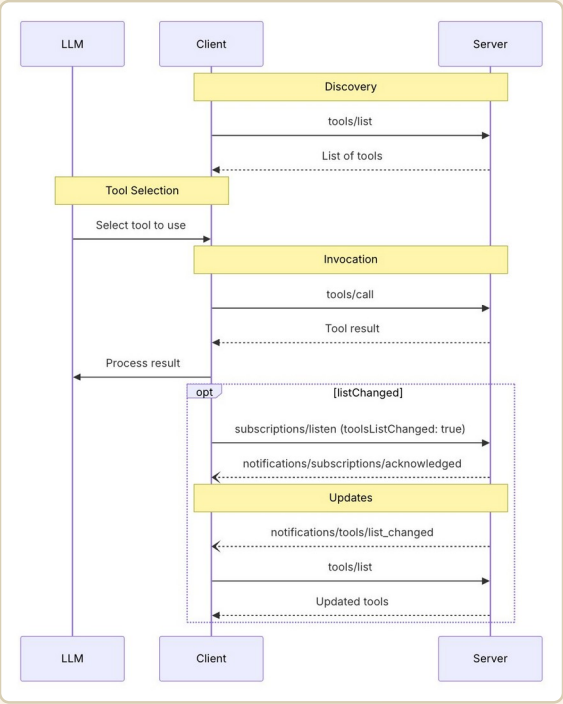
Major changes

1. Remove protocol-level sessions and the `Mcp-Session-Id` header from the Streamable HTTP transport. List endpoints (`tools/list` , `resources/list` , `prompts/list`) no longer vary per-connection. Servers that need cross-call state use explicit, server-minted handles passed as ordinary tool arguments (SEP-2567).
2. Make MCP stateless: remove the `initialize` / `notifications/initialized` handshake. Every request now carries its protocol version and client capabilities in `_meta` (`io.modelcontextprotocol/protocolVersion` , `io.modelcontextprotocol/clientCapabilities`). Clients SHOULD identify themselves on each request (`io.modelcontextprotocol/clientInfo`), and servers SHOULD

Spec 2026-07-28 · Key Changes

Why: load balancers, serverless and caching just work.

A typical conversation



Spec 2026-07-28 · Tools › Message Flow

- **tools/list**
  The client learns which tools exist
- **The LLM picks one**
  From names, descriptions and schemas
- **tools/call**
  The server runs the real function
- **Result → LLM**
  Back as context: the loop of lesson 10
- **subscriptions/listen**
  Opt-in: notified when the list changes

Primitives

Tools · Resources · Prompts · Elicitation

IV

What a server can offer

Tools

Functions the LLM calls

WHO DECIDES

The model

```
search_cars(...)
```

Resources

Read-only data for context

WHO DECIDES

The application

```
estoque://schema
```

Prompts

Templates the user invokes

WHO DECIDES

The user

```
/recomendar
```

Same server, three kinds of control.

Tools, in detail

`name`

Unique id · Required

`title`

Display name

`description`

When to use it · Required

`inputSchema`

JSON Schema of the args · Required

`outputSchema`

Shape of the result

`annotations`

Behavior hints

`annotations`

- **`readOnlyHint`** does not change anything
- **`destructiveHint`** may delete or overwrite
- **`idempotentHint`** safe to repeat
- **`openWorldHint`** talks to the outside world

Hints, not guarantees: clients MUST treat annotations as untrusted unless the server is trusted.

Resources: context by address

Direct

`estoque://schema`

The columns of the stock table

Template

`estoque://cars/{car_id}`

One URI per car — c082, c109, ...

`resources/list` `resources/templates/list` `resources/read`

The application decides what to attach: a file picker, automatic inclusion, or a search with embeddings — your RAG from weeks 08–09 fits right here.

Prompts: templates the user invokes

```python
@mcp.prompt
def recomendar(orcamento: int,
               carroceria: str):
    return (f"Quero um {carroceria}"
            f" até R$ {orcamento}."
            " Busque no estoque e"
            " cite o car_id.")
```



In the host

`/recomendar 80000 SUV`

“Quero um SUV até R$ 80000. Busque no estoque e cite o car_id.”

`prompts/list` `prompts/get`

The server author knows best how to ask for its tools.

And the client side?



Elicitation

The server asks the user for input mid-task

Active



Roots

Folders the server may touch

Deprecated 2026 → pass paths as arguments



Sampling

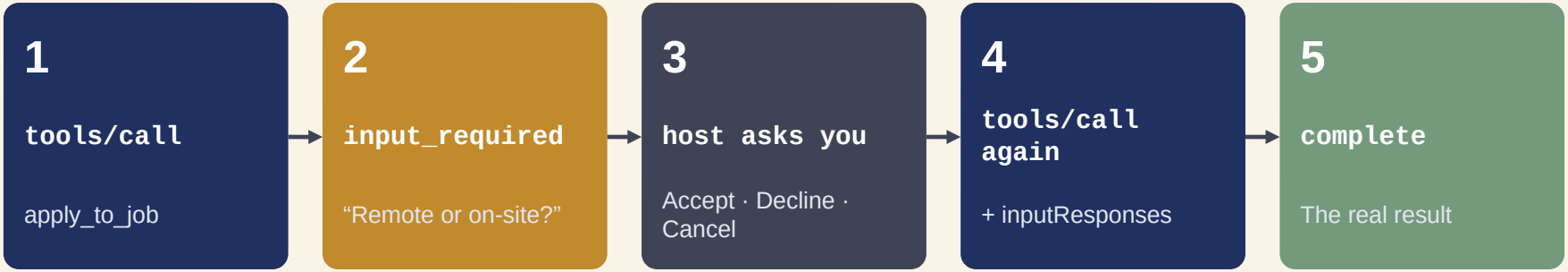
The server borrows the host's LLM

Deprecated 2026 → call an LLM API directly

Old tutorials teach all three. **Today, only elicitation is recommended.**

Elicitation, step by step

A job-application tool needs one answer from you



Never for passwords or secrets. **Servers must not elicit sensitive data, and the user can always say no.**

Build one

FastMCP · The Inspector · Any host



From lesson 10 to an MCP server

LESSON 10 · BY HAND

```
SEARCH_SCHEMA = {"type": "function",
 "function": {
   "name": "search_cars",
   "description": "Search the...",
   "parameters": {"type": "object",
     "properties": {
       "max_price": {"type": ...},
       "body": {"type": ...}},
     "required": ["max_price"]}}}
FUNCTIONS = {"search_cars": ...}
```

TODAY · FASTMCP

```
from fastmcp import FastMCP
mcp = FastMCP("estoque-natal")

@mcp.tool
def search_cars(max_price: float,
                body: str | None = None):
    """Search the used-car stock..."""
    ...  # same pandas code

mcp.run()   # stdio by default
```

One decorator replaces the schema, the dispatch table and the protocol.

A whole server, three primitives

server.py — runs as is · fastmcp 4

```python
mcp = FastMCP("estoque-natal")

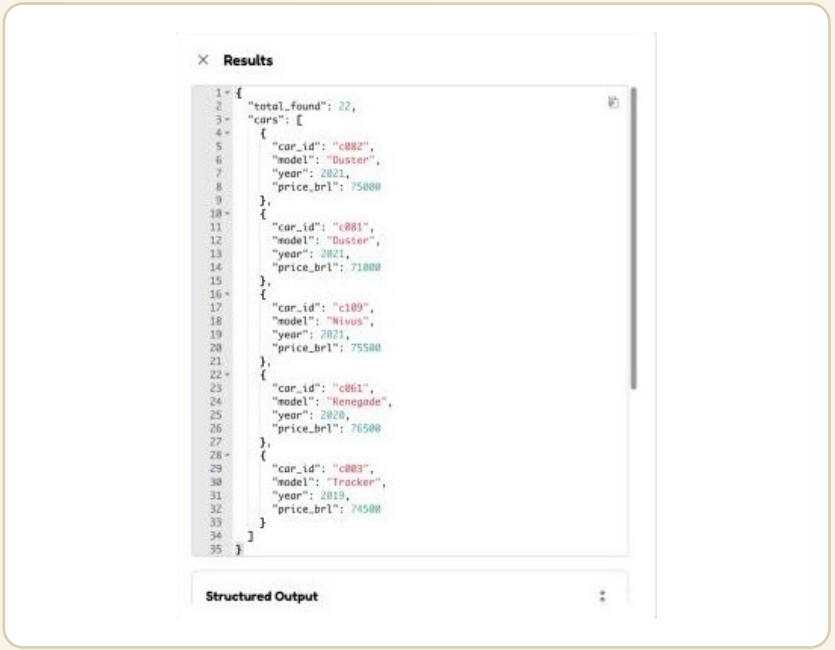
@mcp.tool
def search_cars(max_price: float, body: str | None = None) -> dict:
    """Search the used-car stock by budget and body type."""

@mcp.resource("estoque://schema")
def schema() -> str:
    return ", ".join(cars.columns)

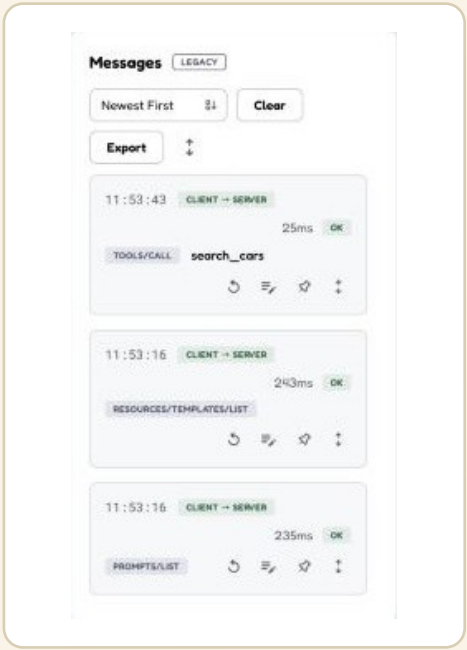
@mcp.prompt
def recomendar(orcamento: int, carroceria: str) -> str: ...
```

- **@mcp.tool** — the model calls it
- **@mcp.resource** — the app reads it
- **@mcp.prompt** — the user invokes it

Test it without any LLM: the Inspector



Tools tab: search_cars(80000, "SUV") → our result

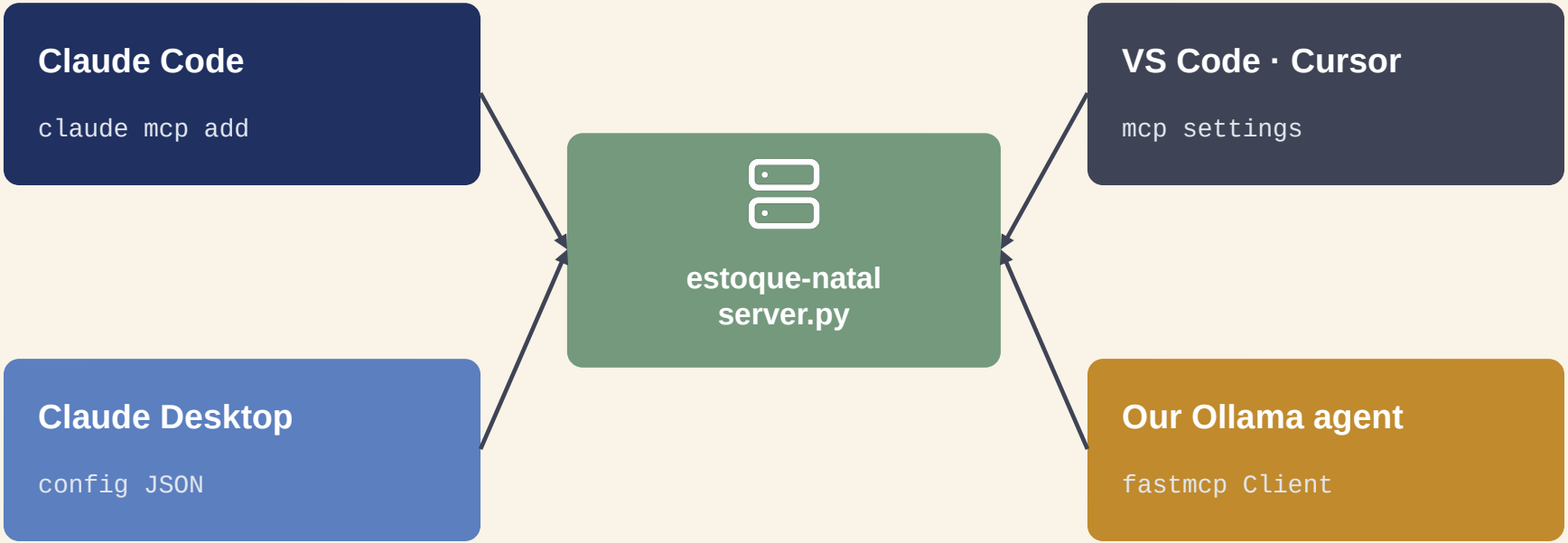


Protocol tab: every message

A form generated from the schema. Fill it, run it, read the JSON.

```
npx @modelcontextprotocol/inspector python server.py
```

Same server, any host



Write once, plug anywhere — that is the M+N promise, made concrete.

lesson11.ipynb — the plan

1 Wrap lesson 10's tools in FastMCP

→

2 Inspect it: list and call

→

3 Talk JSON-RPC by hand

→

4 Connect a Python MCP client

→

5 Give MCP tools to the lesson 10 agent

Same stock, same answer as lesson 10 — through a standard protocol.

`open the notebook — live demo`

Four ideas to keep

M+N, not M×N

One server, any host

Three roles

Host · Client · Server, one client per server

JSON-RPC

Over stdio locally, HTTP remotely

Primitives

Tools · Resources · Prompts

Next: frameworks and multi-agent systems — lesson 12.

References & further reading

Model Context Protocol — Specification 2026-07-28
modelcontextprotocol.io/specification/2026-07-28

MCP Docs — Architecture overview and server concepts
modelcontextprotocol.io/docs/learn/architecture

MCP Inspector
github.com/modelcontextprotocol/inspector

FastMCP 4 — Python framework for MCP servers and clients
gofastmcp.com

JSON-RPC 2.0 Specification
jsonrpc.org/specification

Anthropic (2025) — Donating MCP to the Agentic AI Foundation
blog.modelcontextprotocol.io · Linux Foundation

Next · Nov 9 — Agents III: frameworks, multi-agent and evaluation